

EXPERIMENT - 14

1. Problem Statement

Write a menu-driven C program using `switch-case` to input a positive integer and perform one of the following operations based on the user's choice:

- **Option 1** — Check whether all its digits are unique.
E.g. 1: Number: 12345 → Output: Unique
E.g. 2: Number: 12234 → Output: Not Unique
- **Option 2** — Find the frequency of each digit from `0` to `9`.
E.g. Number: 112333 → `freq_0 = 0`, `freq_1 = 2`, `freq_2 = 1`, `freq_3 = 3`, `freq_4 = 0`, `freq_5 = 0`, `freq_6 = 0`, `freq_7 = 0`, `freq_8 = 0`, `freq_9 = 0`

2. Logic Used

Both operations need the digits of the number one by one. Two statements do all the work:

Statement	Meaning
<code>digit = temp % 10;</code>	Gives the last digit of the number
<code>temp = temp / 10;</code>	Removes the last digit (integer division)

The loop repeats until `temp` becomes 0. For 112333 the digits are extracted in the order **3 → 3 → 3 → 2 → 1 → 1**.

A **counting array** `count[10]` is used for both options. Every time a digit is extracted, `count[digit]` is increased by 1. So `count[0]` stores how many times 0 appeared, `count[1]` how many times 1 appeared, and so on up to `count[9]`.

Option	How the same array is used
1 — Unique check	If any <code>count[digit]</code> becomes greater than 1, that digit has repeated, so the flag <code>unique</code> is set to 0 and the answer is Not Unique .
2 — Frequency	After the loop ends, the array is simply printed from index 0 to 9 as <code>freq_0 ... freq_9</code> .

The array must be set to **all zeros before counting**, otherwise it holds garbage values. Also note that a number can have **at most 10 unique digits** (0–9), so any number with more than 10 digits is

automatically Not Unique.

3. Algorithm

```
Step 1 : START

Step 2 : Display the menu
        1. Check whether all digits are unique
        2. Frequency of each digit (0 to 9)

Step 3 : Read choice

Step 4 : Read num

Step 5 : If num <= 0 then
        Print "Error: not a positive integer"
        Go to Step 10
    End If

Step 6 : Set count[0] to count[9] = 0

Step 7 : switch (choice)

Step 8 : case 1:                                [ UNIQUE CHECK ]
        Set unique = 1, temp = num
        While temp > 0 repeat
            digit = temp % 10
            count[digit] = count[digit] + 1
            If count[digit] > 1 then
                unique = 0
            End If
            temp = temp / 10
        End While
        If unique = 1 then
            Print "Unique"
        Else
            Print "Not Unique"
        End If
        break

Step 9 : case 2:                                [ FREQUENCY ]
        Set temp = num
        While temp > 0 repeat
            digit = temp % 10
            count[digit] = count[digit] + 1
            temp = temp / 10
        End While
        For i = 0 to 9 repeat
            Print "freq_i = count[i]"
        End For
        break

default:
    Print "Invalid choice"
```

break

Step 10 : STOP

4. Test Case Table

TC #	Choice	Input Number	Expected Output	Actual Output	Result
TC1	1	12345	Unique	Unique	PASS
TC2	1	12234	Not Unique	Not Unique	PASS
TC3	2	112333	freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 ... freq_9 = 0	freq_0 = 0, freq_1 = 2, freq_2 = 1, freq_3 = 3, freq_4 ... freq_9 = 0	PASS
TC4	1	7	Unique	Unique	PASS
TC5	2	1020	freq_0 = 2, freq_1 = 1, freq_2 = 1, rest = 0	freq_0 = 2, freq_1 = 1, freq_2 = 1, rest = 0	PASS
TC6	1	1234567890	Unique	Unique	PASS
TC7	1	-5	Error message	Error: -5 is not a positive integer.	PASS
TC8	2	0	Error message	Error: 0 is not a positive integer.	PASS
TC9	3	123	Invalid choice message	Invalid choice! Enter 1 or 2.	PASS

Why these test cases were chosen

TC #	Purpose of the test case
TC1	First example given in the question — normal case, all digits different
TC2	Second example given in the question — a repeated digit (2 appears twice)
TC3	Example given in the question for menu option 2 — verifies the frequency counts
TC4	Boundary case — a single-digit number must be reported as Unique
TC5	Number containing zeros — checks that digit 0 is counted correctly in <code>count[0]</code>
TC6	Maximum case — all ten digits 0–9 used exactly once, still Unique
TC7	Invalid input — a negative number must be rejected

TC #	Purpose of the test case
TC8	Invalid input — zero is not a positive integer, must be rejected
TC9	Invalid menu choice — tests the <code>default:</code> block of the switch statement

5. Source Code

File name: `unique_frequency.c` (75 lines)

```
/* Menu-driven program using switch-case
   1. Check whether all digits of a number are unique
   2. Find the frequency of each digit from 0 to 9          */

#include <stdio.h>

int main()
{
    int choice, digit, i;
    long num, temp;
    int count[10];
    int unique;

    printf("===== MENU =====\n");
    printf("1. Check whether all digits are unique\n");
    printf("2. Frequency of each digit (0 to 9)\n");
    printf("===== \n");
    printf("Enter your choice (1 or 2): ");
    scanf("%d", &choice);

    printf("Enter a positive integer: ");
    scanf("%ld", &num);

    if (num <= 0) {
        printf("Error: %ld is not a positive integer.\n", num);
        return 1;
    }

    for (i = 0; i < 10; i++)
        count[i] = 0;

    switch (choice) {

        case 1:
            temp = num;
            unique = 1;

            while (temp > 0) {
                digit = temp % 10;
                count[digit]++;
                if (count[digit] > 1)
                    unique = 0;
                temp = temp / 10;
            }

            printf("\nNumber : %ld\n", num);
            if (unique == 1)
                printf("Output : Unique\n");
            else
```

```
        printf("Output : Not Unique\n");
        break;

    case 2:
        temp = num;

        while (temp > 0) {
            digit = temp % 10;
            count[digit]++;
            temp = temp / 10;
        }

        printf("\nNumber : %ld\n", num);
        for (i = 0; i < 10; i++)
            printf("freq_%d = %d\n", i, count[i]);
        break;

    default:
        printf("\nInvalid choice! Enter 1 or 2.\n");
        break;
}

return 0;
}
```

6. Writing the Program in the Terminal

The program was written in the `MyAssignment` folder using the `nano` editor:

```
$ cd ~/Desktop/MyAssignment
$ nano unique_frequency.c
```

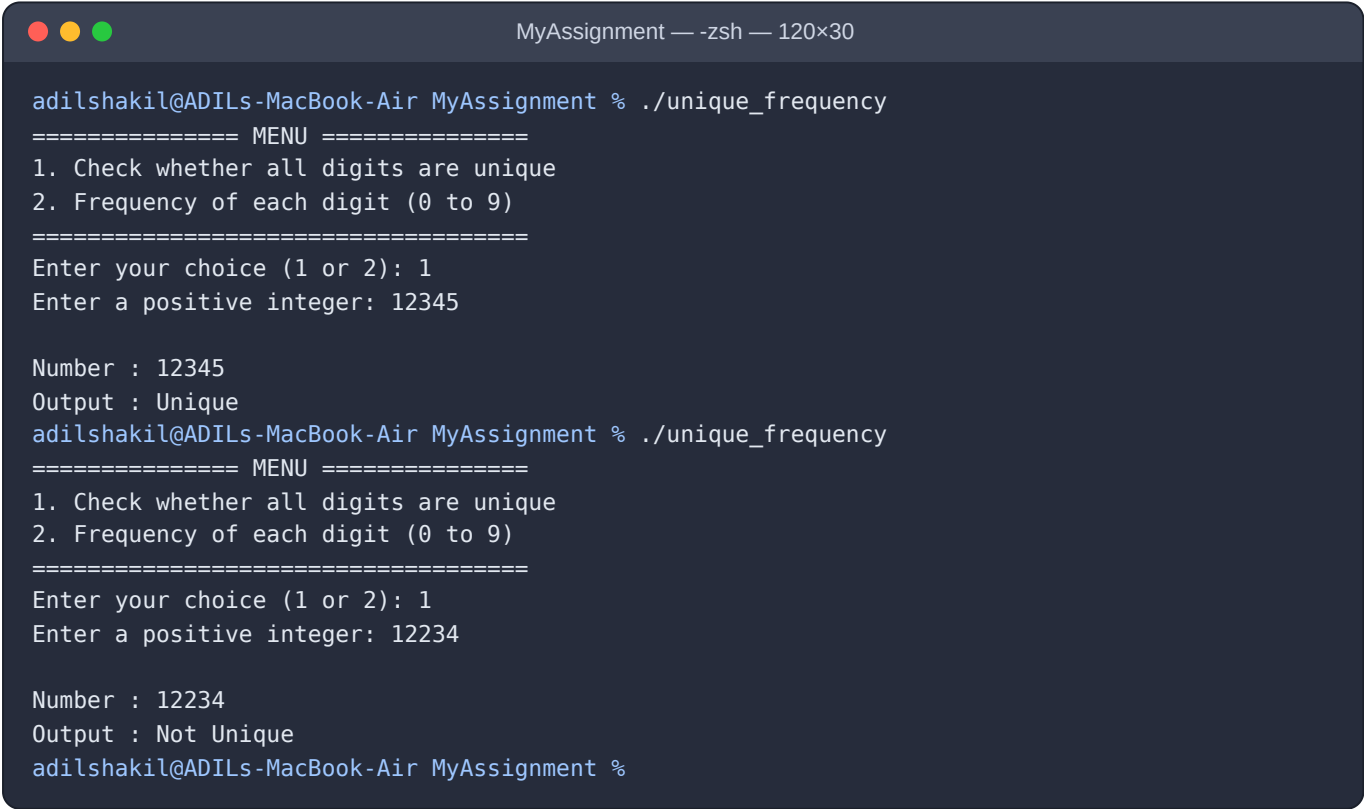
7. Compilation & Execution

The program was compiled with the GCC compiler and executed in the Terminal:

```
$ gcc unique_frequency.c -o unique_frequency
$ ./unique_frequency
```

The program compiled successfully with **no errors and no warnings**.

8. Output



```
MyAssignment — -zsh — 120x30

adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 1
Enter a positive integer: 12345

Number : 12345
Output : Unique
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 1
Enter a positive integer: 12234

Number : 12234
Output : Not Unique
adilshakil@ADILs-MacBook-Air MyAssignment %
```

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 2
Enter a positive integer: 112333

Number : 112333
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
adilshakil@ADILs-MacBook-Air MyAssignment %
```

```
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 2
Enter a positive integer: 1020

Number : 1020
freq_0 = 2
freq_1 = 1
freq_2 = 1
freq_3 = 0
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
adilshakil@ADILs-MacBook-Air MyAssignment %
```



```
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 1
Enter a positive integer: 7

Number : 7
Output : Unique
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 1
Enter a positive integer: 1234567890

Number : 1234567890
Output : Unique
adilshakil@ADILs-MacBook-Air MyAssignment %
```



```
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 1
Enter a positive integer: -5
Error: -5 is not a positive integer.
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 2
Enter a positive integer: 0
Error: 0 is not a positive integer.
adilshakil@ADILs-MacBook-Air MyAssignment % ./unique_frequency
===== MENU =====
1. Check whether all digits are unique
2. Frequency of each digit (0 to 9)
=====
Enter your choice (1 or 2): 3
Enter a positive integer: 123

Invalid choice! Enter 1 or 2.
adilshakil@ADILs-MacBook-Air MyAssignment %
```

9. Conclusion

The menu-driven program was successfully written, compiled and executed using the GCC compiler. All **9 test cases passed**, including the boundary cases (a single-digit number and a number using all ten digits), a number containing zeros, and the two invalid-input cases. The use of `switch-case` allows the required operation to be selected cleanly at run time, and a single counting array `count[10]` is enough to solve both problems — a repeated count greater than 1 answers the uniqueness question, while the same array printed from index 0 to 9 gives the frequency of every digit.